Table 1: Comparing BoT with previous methods across various tasks. We denote the best score in **blue**, and the second-best score in **green**. Our BoT significantly outperforms other methods on all tasks, especially on general reasoning problems.

| Task | Standard | Single-Query | | | Multi-Query | | | **BoT (Ours)** |
|---|---|---|---|---|---|---|---|---|
| | GPT4 [3] | GPT4+CoT [8] | Expert [9] | PAL [10] | ToT [14] | GoT [17] | Meta Prompting [15] | |
| Game of 24 | 3.0 | 11.0 | 3.0 | 64.0 | **74.0** | 73.2 | 67.0 | **82.4** |
| MGSM (avg) | 84.4 | 85.5 | 85.0 | 72.0 | 86.4 | **87.0** | 84.8 | **89.2** |
| Multi-Step Arithmetic | 84.0 | 83.2 | 83.2 | 87.4 | 88.2 | 89.2 | **90.0** | **99.8** |
| WordSorting | 80.4 | 83.6 | 85.2 | 93.2 | 96.4 | 98.4 | **99.6** | **100.0** |
| Python Puzzles | 31.1 | 36.3 | 33.8 | 47.3 | 43.5 | 41.9 | **45.8** | **52.4** |
| Geometric Shapes | 52.6 | 69.2 | 55.2 | 51.2 | 56.8 | 54.2 | **78.2** | **93.6** |
| Checkmate-in-One | 36.4 | 32.8 | 39. 6 | 10.8 | 49.2 | 51.4 | **57.2** | **86.4** |
| Date Understanding | 68.4 | 69.6 | 68.4 | 76.2 | 78.6 | 77.4 | **79.2** | **88.2** |
| Penguins | 71.1 | 73.6 | 75.8 | **93.3** | 84.2 | 85.4 | 88.6 | **94.7** |
| Sonnet Writing | 62.0 | 71.2 | 74.0 | 36.2 | 68.4 | 62.8 | **79.6** | **80.0** |

**Implementation and Baselines** For the fair comparisons with previous methods, we use GPT-4 as the base model of our BoT, including the main experiment and the ablation study (in Section 6). We also use Llama3-8B and Llama3-70B in our analysis part on NVIDIA A100-PCIE-40GB GPU. We compare our Buffer of Thoughts with the following prompting methods: **1. Standard Prompting**: This is our most basic baseline, where an LLM is asked to generate a response directly from the input query, without any specific guiding input-output examples or additional instructions beyond the task description included in the query.

**2. Single-query Method**: This includes Zero-shot CoT [8] and PAL [10], which use the LLM to analyze natural language problems and generate intermediate reasoning steps. We also include Expert Prompting [9], which creates an expert identity tailored to the specific context of the input query, and then integrates this expert profile into the input to generate a well-informed response.

**3. Multi-query Method**: This includes ToT [14] and GoT [17], which enable LLMs to make deliberate decisions by considering multiple reasoning paths and self-evaluating choices to determine the next course of action. These methods also allow for looking ahead or backtracking when necessary to make global decisions. Additionally, we include Meta Prompting [15], which employs an effective scaffolding technique designed to enhance the functionality of LLMs.

### 4.1 BoT Achieves Better Accuracy, Efficiency and Robustness

**Reasoning Accuracy** As shown in Table 1, our BoT consistently outperforms all previous prompting methods across multiple kinds of challenging benchmarks, particularly demonstrated in complicated reasoning tasks such as Game of 24 and Checkmate-in-One. Taking GPT-4 as a baseline, our method achieves an astonishing 79.4% accuracy improvement in Game of 24, and compared to ToT, which has a good performance on this task, we also achieve an 8.4% accuracy improvement. What's more, compared to recent Meta-prompting method [15], we see **significant accuracy improvements: 23% on Game of 24, 20% on Geometric Shapes and 51% on Checkmate-in-One**. Existing methods need complex, iterative, and heuristic search strategies to address these problems on a case-by-case basis. Conversely, our BoT leverages the historical insights and informative guidelines from thought-templates, and further adaptively instantiate a more optimal reasoning structure for addressing these complex problems.

**Reasoning Efficiency** In addition to significant improvements in accuracy, as a multi-query method, our BoT can achieve comparable reasoning time to single-query method across various tasks, while being considerably less than conventional multi-query method like ToT [14] as shown in Figure 3. For example, in Game of 24, both single-query and multi-query methods necessitate iterative and heuristic searches to identify feasible solutions. This process is particularly time-consuming and inefficient, especially for the multi-query method, which involves conducting multi-query search and backtrace phases. In contrast, our BoT directly retrieves a thought-template in code format, thus a program is instantiated to traverse combinations of numbers and symbols, thereby eliminating the need to build the reasoning structure from scratch. This allows for solving the problem with just one query after invoking the problem-distiller, significantly reducing the time required for complex reasoning. Notably, our **BoT requires only 12% of the cost of multi-query methods** (e.g., tree of thoughts and meta-prompting) on average.

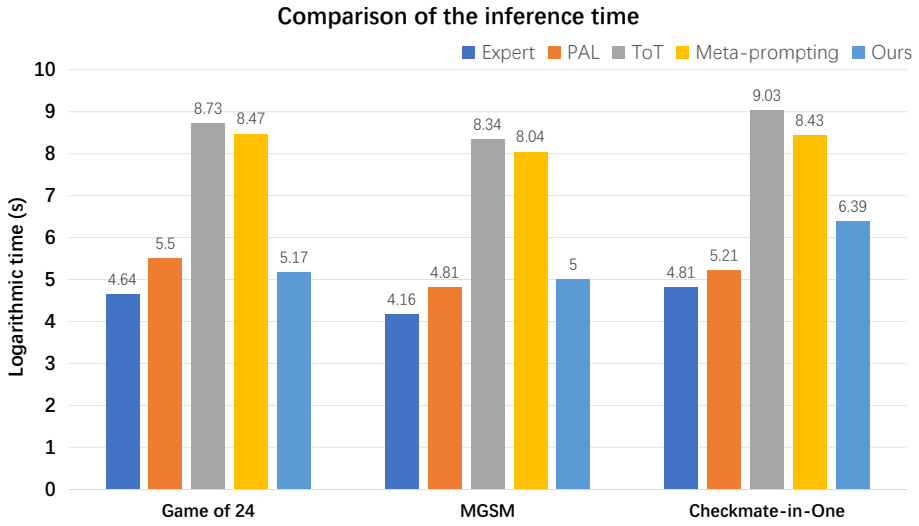


Figure 3: Comparison of **logarithmic inference time** between our Buffer of Thoughts and GPT4 [3], GPT4+CoT [8], Expert-prompting [9], PAL [10], ToT [14] across different benchmarks.

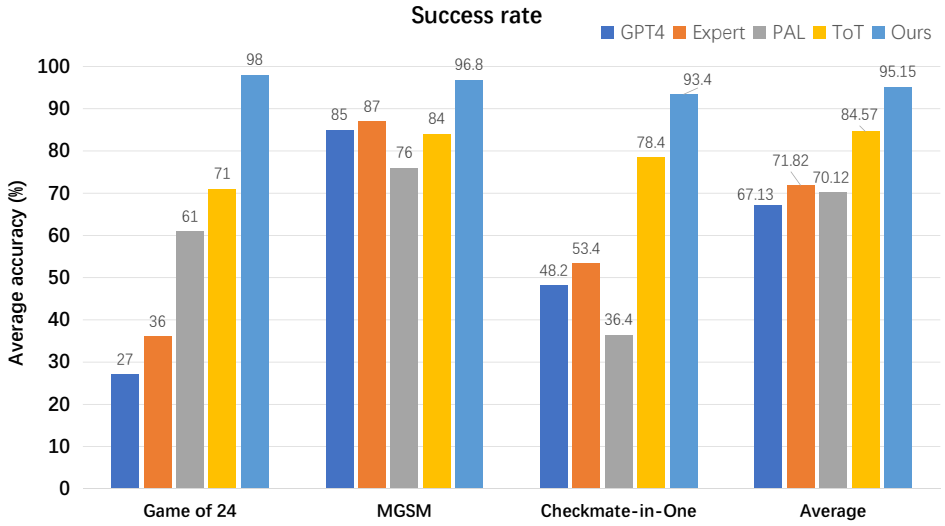


Figure 4: Comparison of reasoning robustness between our Buffer of Thoughts and GPT4 [3], GPT4+CoT [8], Expert-prompting [9], PAL [10], ToT [14] across different benchmarks.

**Reasoning Robustness** To better evaluate our BoT, we devise a new evaluation metric: *success rate*, which is used to assess the reasoning robustness. We randomly sample 1000 examples from various benchmarks as a test subset and evaluate different methods on this subset. As shown in Figure 4, we repeat this evaluation process 10 times and take the average accuracy as the success rate of different methods on each benchmark. Compared with other methods, our BoT consistently maintains a higher success rate across various tasks, surpassing the second-best by 10% in average success rate. We attribute our outstanding robustness to the great generalization ability of our distilled thought-templates during reasoning across different tasks. By offering high-level thought from the suitable thought-templates, the stability of our method across different tasks is greatly enhanced.

## 5 Model Analysis

**Distribution Analysis of Thought-Templates** As depicted in the left figure of Figure 5, we choose six different benchmarks, each sampled with 100 distinct tasks. We update the meta-buffer from scratch, and after completing all sampled tasks, we display the number of derived thought-templates.

We can observe that our BoT generates a greater number of thought-templates in the MGSM tasks that contain more diverse scenarios. In tasks with relatively simple requirements, such as Checkmate-in-One and Penguins, BoT produces more fixed thought-templates tailored for those specific issues. The distribution of templates indicates that our BoT can effectively discover appropriate thought templates for different benchmarks.

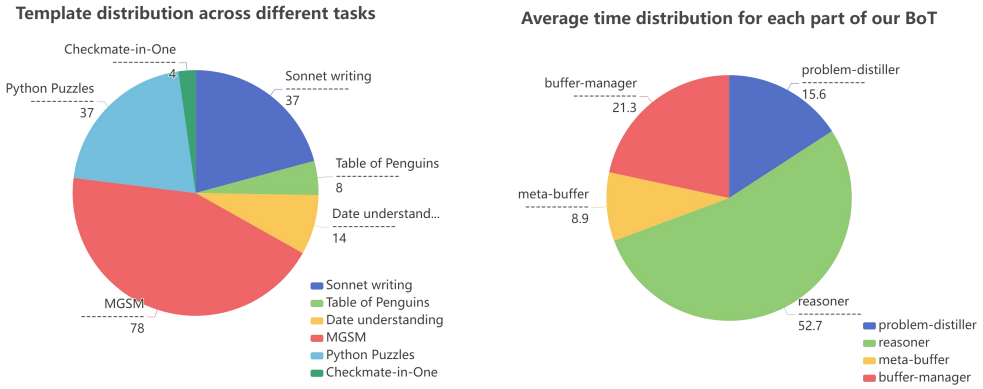


Figure 5: Distribution Analysis of Thought-Templates and Time. *Left*: Distribution Analysis of Thought-Templates. *Right*: Time Distribution of BoT.

**Distribution Analysis of Time Cost** As illustrated in Figure 5, we measured the average time cost for each component of BoT's reasoning framework across different tasks. The time required for distilling task information and template retrieval is relatively short, whereas instantiated reasoning takes longer. Overall, considering the complexity of different components, our BoT achieves a relatively balanced distribution of time cost, demonstrating the efficiency of our BoT framework.

**Better Trade-off between Model Size and Performance** As depicted in Figure 6, on Game of 24, word list sorting and Checkmate-in-One, Llama3-8B and Llama-70B models [6] may result in poor outcomes. However, equipped with our BoT, both models demonstrate a substantial accuracy improvement. Notably, **BoT+Llama3-8B has the potential to surpass single Llama3-70B model**. Our BoT enables smaller models to exhibit the capabilities that approximate or even surpass larger models, significantly bridging the gap between their reasoning abilities. Furthermore, it greatly diminishes the inference cost required by large language models when tackling complex problems.

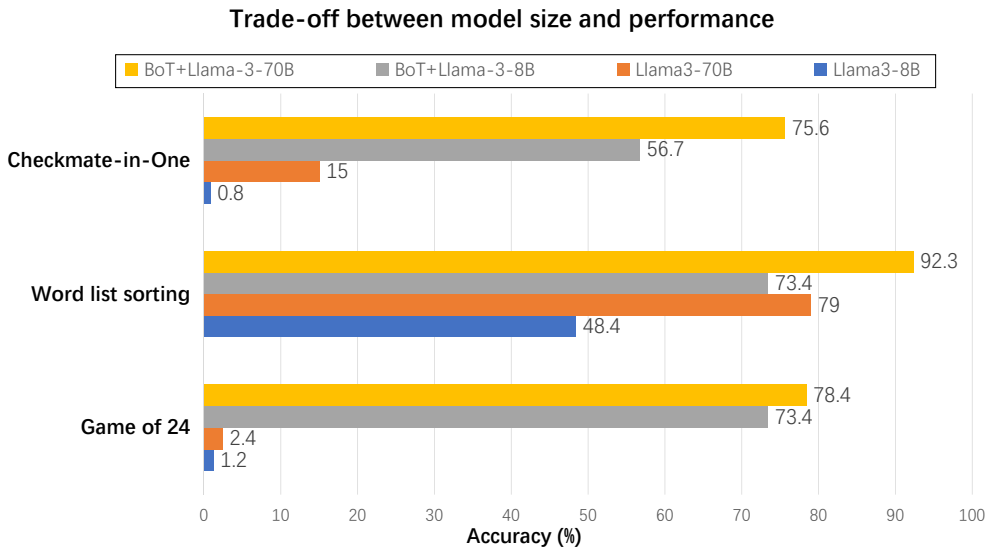


Figure 6: We evaluate the trade-off between model size and performance with Llama3-8B and Llama3-70B models on three challenging benchmarks.